457_a4

2026-03-17

Question 1

Part a)

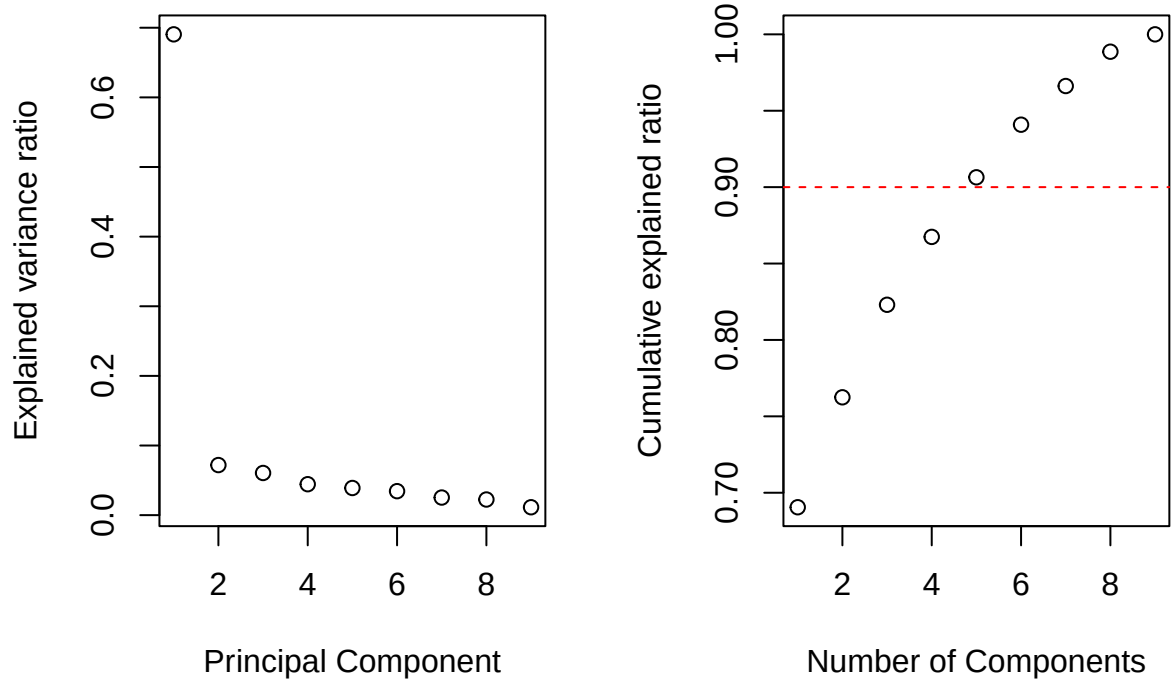
```r
X <- read.csv("A4Q1_X.csv")
Y_df <- read.csv("A4Q1_Y.csv")
Y <- Y_df[[1]] # Extract as a vector

# Perform PCA
tr_pca <- prcomp(X)

# Calculate explained variance ratio
ex_var <- tr_pca$sdev^2 / sum(tr_pca$sdev^2)

# Plot the explained variance ratio
par(mfrow = c(1, 2))
plot(ex_var,
     xlab = "Principal Component",
     ylab = "Explained variance ratio")

# Plot the cumulative explained variance ratio
cum_var <- cumsum(ex_var)
plot(cum_var,
     xlab = "Number of Components",
     ylab = "Cumulative explained ratio")
abline(h = 0.9, col = "red", lty = 2) # Threshold line at 90%
```




```r
# Display result
print("The smallest number of components to exceed 90% explained variance is:")
```

```
## [1] "The smallest number of components to exceed 90% explained variance is:"
```

```r
(k_90 <- which(cum_var > 0.90)[1])
```

```
## [1] 5
```

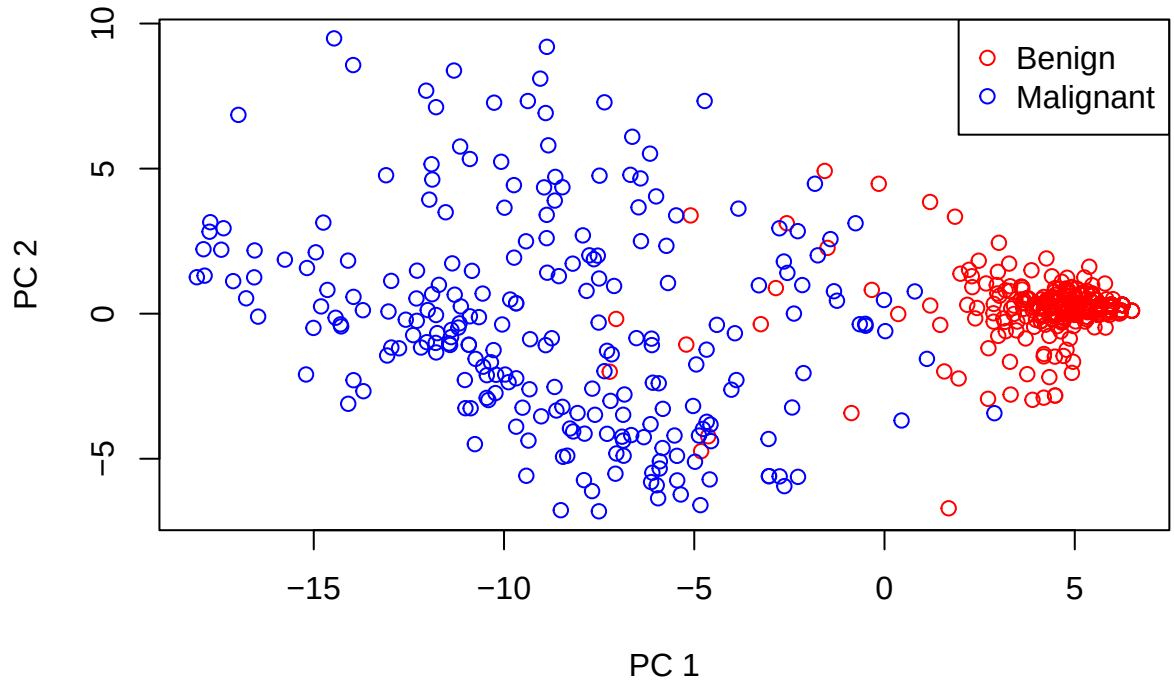
Part b)

```r
par(mfrow = c(1, 1))

# Plot the first two principal components
# "benign" in red, "malignant" in blue
plot(tr_pca$x[Y == "benign", 1], tr_pca$x[Y == "benign", 2],
     col = "red",
     pch = 1,
     xlim = range(tr_pca$x[, 1]),
     ylim = range(tr_pca$x[, 2]),
     xlab = "PC 1", ylab = "PC 2")
```

```r
points(tr_pca$x[Y == "malignant", 1], tr_pca$x[Y == "malignant", 2],
       col = "blue",
       pch = 1)

# Add a legend for clarity
legend("topright", legend = c("Benign", "Malignant"),
       col = c("red", "blue"), pch = 1)
```



2. $U \in \mathbb{R}^{p \times p}$ $\quad UU^T = I_{p\times p}$ (orthonormal)

Show $\underbrace{\left(I_{p\times p} - \sum_{j=1}^{k} U_{\cdot,j} U^T_{\cdot,j}\right)}_{\text{Let this part be } A} U = [0_{p\times k}, U_{\cdot,k+1}, \dots, U_{\cdot,p}]$

$$A U_{\cdot,i} = \left(I_{p\times p} - \sum_{j=1}^{k} U_{\cdot,j} U^T_{\cdot,j}\right) U_{\cdot,i}$$

ith column

$$= I_{p\times p} U_{\cdot,i} - \sum_{j=1}^{k} \left(U_{\cdot,j} U^T_{\cdot,j}\right) U_{\cdot,i}$$

Case $1 \le i \le k$

Because by orthogonality property $UU^T = I \Rightarrow U^T_{\cdot,j} \cdot U_{\cdot,i} = 0$ for $i \neq j$

and because U is orthonormal $\quad U^T_{\cdot,j} \cdot U_{\cdot,j} = 1$ for $i = j$

So $\sum_{j=1}^{k} U_{\cdot,j} \left(U^T_{\cdot,j} U_{\cdot,i}\right)$

$= U_{\cdot,i}$ since only when $i = j$ will this summation remain with exactly 1 vector.

So $A U_{\cdot,i} = I_{p\times p} U_{\cdot,i} - U_{\cdot i} = \vec{0}$

The first k columns are all 0.

Case $k+1 \le i \le p$

$A U_{\cdot,i} = I_{p\times p} U_{\cdot i} - \sum_{j=1}^{k} U_{\cdot j} (\underline{0})$ because in no cases $i = j$ when $i \ge k+1$ and $j \le k$

$= U_{\cdot,i}$

so 2 orthogonal vectors multiply and becomes 0.

$$\therefore A U = \underbrace{\begin{array}{c} \\ p \text{ rows} \end{array}\begin{bmatrix} \overbrace{\begin{matrix} 0 & 0 & 0 \\ 0 & \vdots & \vdots \\ 0 & \vdots & \vdots \\ 0 & \vdots & \vdots \\ 0 & \vdots & \vdots \end{matrix}}^{k \text{ colums}} & U_{\cdot,k+1}, U_{\cdot,k+2}, \dots, U_{\cdot,p} \end{bmatrix}}_{p \text{ rows}}$$

as requested.

3. a) $(I_{p\times p} - U_{\cdot,1} U_{\cdot 1}^T)\, \hat{\Sigma}\, (I_{p\times p} - U_{\cdot,1} U_{\cdot 1}^T)$

$$\underline{(I_{p\times p} - U_{\cdot,1} U_{\cdot 1}^T)} \cdot U \cdot \Lambda \cdot U^T (I_{p\times p} - U_{\cdot,1} U_{\cdot 1}^T)$$

Notice this part is basically question 2 where $k=1$

So this is a $p\times p$ matrix that looks like $[\vec{0} \quad U_{\cdot 2} \quad U_{\cdot 3} \quad \dots \quad U_{\cdot p}]$

Let this $I - U_{\cdot 1} \cdot U_{\cdot 1}^T$ matrix be represented as P.

We know $\hat{\Sigma} = U \Lambda U^T$

λ_1 multiplied to row 1
λ_2 — — — row 2 ...

$$= U_{\cdot 1}(\lambda_1 U_{\cdot 1}^T) + U_{\cdot 2}(\lambda_2 U_{\cdot 2}^T) + \dots$$

$$= \sum_{k=1}^{p} \lambda_k U_{\cdot k} U_{\cdot k}^T$$

So $P\left(\sum_{k=1}^{p} \lambda_k U_{\cdot k} U_{\cdot k}^T\right) P$

$$= \sum_{k=1}^{p} \lambda_k (P U_{\cdot k})(U_{\cdot k}^T P)$$

We know when $k=1$ $P U_{\cdot 1} = 0$ so $\lambda_1 P U_{\cdot 1} U_{\cdot 1}^T P = 0$

when $k>1$ $P U_{\cdot,k} = U_{\cdot k}$, $U_{\cdot,k}^T P = U_{\cdot,k}^T P^T = (P U_{\cdot,k})^T$

because P is symmetric

$= U_{\cdot,k}^T$

So this equals $0 + \sum_{k=2}^{p} \lambda_k U_{\cdot k} U_{\cdot,k}^T$

b) Already shown during the process of showing for 3 a)

$$U \begin{bmatrix} 0 & \cdots & 0 \\ \vdots & \lambda_2 & \vdots \\ \vdots & & \ddots \\ 0 & 0 \cdots & \lambda_p \end{bmatrix} U^T = 0 \cdot U_{\cdot,1} U_{\cdot 1}^T + \lambda_2 U_{\cdot,2} U_{\cdot 2}^T + \dots + \lambda_p U_{\cdot p} U_{\cdot p}^T$$

$$= \sum_{k=2}^{p} \lambda_k U_{\cdot,k} U_{\cdot k}^T$$

c) $\hat{x}_i = (x_i^T U_{\cdot,1}) U_{\cdot,1}$

$$x_i - \hat{x}_i = x_i - \underbrace{x_i^T U_{\cdot,i}}_{\text{scalar}} U_{\cdot,i}$$

$$= x_i - \underbrace{U_{\cdot,i}}_{\text{dot product}} \underbrace{x_i^T U_{\cdot,i}}$$

$$= x_i - U_{\cdot,i} U_{\cdot,i}^T x_i$$

$$= \underbrace{(I - U_{\cdot,i} U_{\cdot,i}^T)}_{\text{let this be } P\text{, same as part a)}} x_i$$

$$= P x_i$$

Recall $P^T = (I - U_{\cdot,1} U_{\cdot,1}^T)^T$

$= I^T - (U_{\cdot,1} U_{\cdot,1}^T)^T$

$= I - (U_{\cdot,1}^T)^T U_{\cdot,1}^T$

$= I - U_{\cdot,1} \cdot U_{\cdot,1}^T$

$= P$

Now

$$\|x_i - \hat{x}_i\|^2 = (P x_i)^T (P x_i)$$

$$= x_i^T P^T P x_i$$

$$= x_i^T (P)^2 x_i \quad \text{because } P \text{ is symmetric and idempotent}$$

$$= x_i^T P x_i$$

$1\times p \cdot p\times p \cdot p\times 1 \rightarrow$ scalar

$$= \text{trace}(x_i^T P x_i)$$

$$= \text{trace}(P x_i x_i^T)$$

$$= \text{trace}(P^2 x_i x_i^T) \quad P = P^2$$

$$= \text{trace}(P x_i x_i^T P)$$

$$= \text{trace}\left((I - U_{\cdot,1} U_{\cdot,1}^T) x_i x_i^T (I - U_{\cdot,1} U_{\cdot,1}^T)\right)$$

and $P^2 = (I - U_{\cdot,1} U_{\cdot,1}^T)^2$

$= I \cdot I - I U_{\cdot,1} U_{\cdot,1}^T - U_{\cdot,1} U_{\cdot,1}^T I + (U_{\cdot,1} \underbrace{U_{\cdot,1}^T)(U_{\cdot,1}}_{=1 \text{ orthogonal}} U_{\cdot,1}^T)$

$= I - U_{\cdot,1} U_{\cdot,1}^T - U_{\cdot,1} U_{\cdot,1}^T + U_{\cdot,1} U_{\cdot,1}^T$

$= I - U_{\cdot,1} U_{\cdot,1}^T = P$

So

$$\frac{1}{n} \sum_{i=1}^{n} \|x_i - \hat{x}_i\|^2 = \frac{1}{n} \sum_{i=1}^{n} \text{trace}(P x_i x_i^T P)$$

$$= \text{trace}\left(P \left(\frac{1}{n} \sum_{i=1}^{n} x_i x_i^T\right) P\right)$$

by definition $\hat{\Sigma} = n^{-1} X^T X = \frac{1}{n} \sum_{i=1}^{n} x_i x_i^T$

$$= \text{trace}(P \hat{\Sigma} P)$$

$$= \text{trace}\left(\sum_{k=2}^{p} \lambda_k U_{\cdot,k} U_{\cdot,k}^T\right) = \sum_{k=2}^{p} \lambda_k \underbrace{\text{trace}(\underbrace{U_{\cdot,k} U_{\cdot,k}^T}_{=1})}_{=1}$$

$$\boxed{= \sum_{k=2}^{p} \lambda_k}$$

4. a) $\phi(\beta) = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\sigma(x_i^T\beta)) + (1-y_i)\log(1-\sigma(x_i^T\beta))\right) + \frac{\lambda}{2}\beta^T\beta.$

$\frac{d}{dz}\sigma(z) = \sigma(z)(1-\sigma(z))$ $\quad \frac{d}{dz}\log(\sigma(z)) = 1-\sigma(z)$ $\quad \frac{d}{dz}\log(1-\sigma(z)) = -\sigma(z)$

$\frac{d}{d\beta} x_i^T\beta = x_i$

$$\nabla\phi(\beta) = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i\left(1-\sigma(x_i^T\beta)\right)\cdot x_i + (1-y_i)(-\sigma(x_i^T\beta))(x_i)\right) + \lambda\beta$$

$$= -\frac{1}{n}\sum_{i=1}^{n}\left(\left[y_i - \cancel{y_i\sigma(x_i^T\beta)} + \left(-\sigma(x_i^T\beta) + \cancel{y_i\sigma(x_i^T\beta)}\right)\right]x_i\right) + \lambda\beta$$

$$= -\frac{1}{n}\sum_{i=1}^{n}\left(\left[y_i - \sigma(x_i^T\beta)\right]x_i\right) + \lambda\beta$$

or $$= \frac{1}{n}\sum_{i=1}^{n}\left(\sigma(x_i^T\beta)x_i - y_i\right) + \lambda\beta$$

$$H_\phi(\beta) = \frac{d}{d\beta}\nabla\phi(\beta)$$

$$= \frac{1}{n}\sum_{i=1}^{n}\left(\sigma(x_i^T\beta)\left(1-\sigma(x_i^T\beta)\right)\cdot x_i x_i^T\right) + \lambda I_{p\times p}$$

b) Yes it is.

$\sigma(x_i^T\beta)(1-\sigma(x_i^T\beta))$: always positive (≥ 0) because $\sigma(z)$ outputs value between $0 \sim 1$.

$x_i x_i^T$: for any zero vector v, $v^T(x_i x_i^T)v = (v^T x_i)^2 \geq 0$

So this part is also ≥ 0.

$\frac{1}{n}\sum_{i=1}^{n}(+ve)$: positive summation.

If $\lambda = 0$, then the objective is positive semi-definite

if $\lambda > 0$, — — — — positive definite.

Thus becaue Hession is PSD, the function is convex.

We are guaranteed to find the local minimum = global minimum.

Question 4

Part c)

```r
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```r
X <- as.matrix(read.csv("A4Q4_X.csv"))
Y <- as.vector(read.csv("A4Q4_Y.csv")[[1]])

# Define objective function
NR_logistic_reg <- function(X, Y, lambda = 1, maxIter = 100, epsilon = 1e-10) {
  n <- nrow(X)
  p <- ncol(X)

  # Initialize beta at zero
  beta_nr <- rep(0, p)

  for (i in 1:maxIter) {
    beta_old <- beta_nr

    # Calculate probabilities: sigma(X %*% beta)
    eta <- X %*% beta_old
    p_vec <- as.vector(exp(eta) / (1 + exp(eta)))

    # Calculate the Gradient
    grad <- (1/n) * t(X) %*% (p_vec - Y) + lambda * beta_old

    # Calculate the Hessian
    W_diag <- p_vec * (1 - p_vec)
    Hessian <- (1/n) * t(X) %*% (W_diag * X) + lambda * diag(p)

    # Newton-Raphson Update: beta_new = beta_old - H^-1 * grad
    beta_nr <- beta_old - solve(Hessian) %*% grad

    # Check for convergence using the mean squared difference
    if (mean((beta_nr - beta_old)^2) < epsilon) {
      print("Algorithm converged, iteration:")
      print(i)
      break
    }
  }
  return(as.vector(beta_nr))
}

# Compute beta_hat(1) using manually written algorithm
lambda_val <- 1
beta_hat_1 <- NR_logistic_reg(X, Y, lambda = lambda_val)
```

```
## [1] "Algorithm converged, iteration:"
## [1] 3
```

```r
# Compute beta_hat(2) using glmnet
# alpha = 0 is Ridge (L2), intercept = FALSE
fit_glmnet <- glmnet(X, Y, family = "binomial", alpha = 0,
                     lambda = lambda_val, intercept = FALSE, standardize = FALSE)
# Remove intercept index
beta_hat_2 <- as.vector(coef(fit_glmnet, s = lambda_val))[-1]

# Results
print(beta_hat_1)
```

```
##  [1] -0.093219144  0.200656657  0.054353170 -0.002654478 -0.095312309
##  [6]  0.124965546 -0.014332990 -0.005567823 -0.035824216  0.014958421
## [11]  0.024481190 -0.001147721 -0.025053599 -0.018943283  0.011258497
```

```r
print(beta_hat_2)
```

```
##  [1] -0.093219144  0.200656658  0.054353169 -0.002654478 -0.095312309
##  [6]  0.124965546 -0.014332990 -0.005567823 -0.035824216  0.014958421
## [11]  0.024481190 -0.001147721 -0.025053599 -0.018943283  0.011258497
```

```r
comparison_error <- mean((beta_hat_1 - beta_hat_2)^2)

print(comparison_error)
```

```
## [1] 1.15568e-19
```

```r
print(comparison_error < 10^-8)
```

```
## [1] TRUE
```